

PRODUCT OVERVIEW · MAY 2026 · CONFIDENTIAL

PlanMyAgents

The universal trust and marketplace layer for AI agents

We index AI agents, MCP servers, A2A agents, and AI-native services. We rank them with evidence and benchmark gates. We hand users honest recipes — runnable when a provider is tested and configured, and refused with reasons when it is not. Built for a future where AI agents themselves call this layer to discover, vet, and orchestrate the agents they need to complete a goal.

The shift: chat → agents → fragmented supply

LAYER	WHAT'S EMERGING	TODAY'S REALITY	
Tool protocol	MCP (Anthropic, Nov 2024)	1,000+ servers on Smithery alone, doubling quarterly	
Agent interop	A2A (Google et al, 2025)	200+ public agent cards, no quality ranking	
AI-native services	Perplexity, Linkup, Exa, Apify, Browserbase	Programmatic execution, but unbenchmarked	
LLM hosts with native tool use	Claude Desktop, Cursor, Cline, ChatGPT, Gemini	Every host integrates differently; no shared trust layer	
Workflow tools adopting agents	n8n, Zapier, Make adding AI nodes	Need agent recommendations they cannot produce themselves	

The supply is exploding. The trust and recommendation layer for it is empty.

The problem — both sides

Buyer pain. A developer wiring Claude Desktop, Cursor, or n8n hits the same wall: *"There are 200 MCPs that claim to do web search. Which is real? Which works with my host? What is my fallback when this one breaks?"*

Today's only options are hand-searching Smithery/Reddit/GitHub, trusting the first result, asking an LLM that hallucinates tool URLs, or building an internal directory that never stays maintained.

Vendor pain. Agent vendors have nowhere neutral to be discovered, ranked, or benchmarked. They submit to seven catalogs and get ranked nowhere. They cannot tell buyers whether their agent actually works without a neutral authority. They have no signal of what categories buyers are asking for.

Missing layer: *which agent verifiably does this sub-task, with what evidence, at what cost — and where do trusted vendors get found and benchmarked?*

Our wedge — Plan → Discover → Recipe (with honest refusal)

```
USER GOAL
↓
[ PLANNING ]      LLM goal decomposer + label reconciler
↓
[ DISCOVERY ]     17 scout sources → pgvector index → trust ladder → LLM judge
↓
[ PACKAGING ]     Ranker → recipe (Claude / Cursor / n8n / markdown / CLI)
↓
[ HANDOFF ]       User runs in their host with their credentials
↓
[ FEEDBACK ]      Refusals → demand events; executions → benchmark data
                  Public APIs: /discovery/index-freshness · /discovery-gaps · /open-mcp-opportunities
```

Honesty is the product feature. If no provider is tested, runnable, and inside cost gates, we refuse with reasons rather than fabricate a recipe.

The feedback loop is itself a public API surface. `/discovery/index-freshness` reports per-source freshness, `/discovery-gaps` exposes the per-capability rollup of unmet `/goal` requests, and `/open-mcp-opportunities` lists capabilities where an OpenAPI spec exists but no agent wraps it. These are the demand-signal moat — and they ship today.

Discovery + ranking + benchmark is the proprietary engine. Recipes + the future marketplace are how we monetize it.

Current state of the product

AREA	STATUS TODAY
Backend	FastAPI + Postgres + pgvector; in-process execution; cost caps; refusal-with-reasons
Frontend	Next.js 16; goal flow, categories, agent detail, search, leaderboards, gaps
Discovery	17 live scouts → pgvector semantic index → normalizer → dedupe → LLM candidate judge → 5-tier verification. Sources: 5 curated manifests (98 catalog seeds across MCP / A2A / AI-agent / web-docs) + 12 remote: 4 MCP-aggregator catalogs (official MCP Registry, MCP Marketplace, Smithery, Glama — combined indexed surface ~30,000 servers) + 1 npm package-registry scout (~47K MCP-tagged packages) + 3 GitHub scouts (code search, recently-pushed, awesome-* list scraper) + APIs.guru, HN agent-watcher, Vendor RSS, Moltbook
Trust model	5-tier evidence ladder: <code>capability_verified</code> > <code>registered_in_directory</code> > <code>known_provider</code> > <code>community_listed</code> > <code>unverified</code> ; migrated and consistent across code, DB, UI
Recipes	5 export formats round-trip-verified end-to-end (<code>docs/manual-test-log.md</code>). MCP-shape (Claude / Cursor) spawn real npm-published servers and complete the JSON-RPC <code>initialize</code> handshake; HTTP-shape (n8n / CLI) now use registry-authoritative endpoint, method, and auth-header wiring with an <code>npm view</code> resolvability gate before emit
Benchmarks	Three routable cells in <code>benchmark_runs</code> : (1) Razorpay <code>payment_authorization</code> — live run on 2026-05-15, 5/5 at 1.00 quality, 178-265 ms, replayable from <code>agents.json:benchmark_status_evidence</code> ; (2) Resend <code>email_send</code> — 5/5 against the published <code>@resend.dev</code> synthetic addresses; (3) Firecrawl <code>web_scraping</code> — 5/5 against RFC-2606 reserved domains. The Resend and Firecrawl rows are tagged <code>output._provenance = "response_fixture_pending_live_key"</code> — the wrapper is exercised end-to-end via a deterministic transport, and the cells flip to true live runs the moment <code>RESEND_API_KEY</code> / <code>FIRECRAWL_API_KEY</code> reach the cron host. Re-runs nightly via <code>make benchmark-cron</code> ; a GitHub Actions guard (<code>evidence-health-guard.yml</code>) fails <code>main</code> if any of the three live counters drops to zero. 25 hand-authored cases across 5 capabilities (5 each for <code>payment_authorization</code> , <code>email_send</code> , <code>web_scraping</code> , <code>shipping_quote</code> , <code>price_comparison</code>) — adversarial cases verify wrappers can't hallucinate ids or content
Live evidence layer	Public <code>/health/evidence</code> + <code>/evidence/recent-verifications</code> endpoints back a homepage strip and a "Recent verifications" panel on <code>/discovery-gaps</code> and <code>/open-mcp-opportunities</code> . Five evidence tables (<code>benchmark_runs</code> , <code>verification_records</code> , <code>discovery_run_events</code> , <code>capability_demand_events</code> , <code>discovery_gap_events</code>) are populated and queried live — every count above the fold is a Postgres row, not a slide
Registry	15 hand-curated provider entries across 26 capability definitions (Stripe, Razorpay, Resend, Firecrawl, Shippo, eBay, Hunter, Apollo, ...), fed by 98 curated MCP / A2A / AI-agent / web-doc catalog seeds , with 282 live-discovered candidates indexed in Postgres
Pro tier	Shipped — Clerk auth (373 LOC in <code>apps/api/planmyagents_api/auth/clerk.py</code>), Stripe checkout + webhook (565 LOC in <code>apps/api/planmyagents_api/billing/</code>), workspaces + roles + saved recipes (1,070 LOC in <code>apps/api/planmyagents_api/marketplace_store/</code>). <code>/billing/checkout</code> + <code>/stripe/webhook</code> live in OpenAPI; saved-recipes round-trip end-to-end
Hosting plan	AWS path documented: EC2 + RDS + CloudFront + Route 53 for <code>planmyagents.com</code>

Three-phase product ladder

PHASE	WHAT WE OFFER	CUSTOMER	STATUS
1 — Plan + Discover + Recipe <i>(now → Y1)</i>	Free discovery, free recipe exports, Pro tier (Clerk auth + Stripe billing + workspaces + saved recipes) shipped as a skeleton today	Developers, operators	Building today — 3 routable benchmark cells (Razorpay live, Resend + Firecrawl response-fixture), 15 hand-curated providers / 26 capabilities / 98 catalog seeds / 282 live-discovered candidates, 17 scouts on a 24h cron , evidence-health CI guard on <code>main</code>
2 — Vendor marketplace <i>(Y1 end → Y2)</i>	Claimed profiles, vendor-funded benchmarks, sponsored placement (disclosed, never above natural #1), demand-data API	Agent vendors	Designed; not yet built
3 — Partnership platform <i>(Y2+)</i>	OEM into workflow hosts (n8n, Zapier), LLM hosts (Cursor, Cline, Claude Desktop), and cloud platforms; private enterprise registries	Workflow / LLM hosts, enterprise	Planned

Each phase de-risks the next. Phase 1 builds index density and user trust. Phase 2 monetizes vendors after Phase 1 traffic is real. Phase 3 captures distribution after Phase 2 brand exists.

Phase 1 surfaces (what exists or is closest to existing)

SURFACE	USER	WHAT THEY GET TODAY	PRICING
/discovery/search	Developer	Ranked agent list with evidence + benchmark + runnability badges	Free
/goal → recipe	Developer / operator	Goal decomposed, providers matched, recipe exported when steps are runnable, refused with a gap-only runbook and a "Recipe coverage: X / N exportable steps" badge when not	Free
/leaderboards/{cap}	Buyer / vendor	Per-capability ranking with credibility band (synthetic → smoke → developing → publishable)	Free
/open-mcp-opportunities	Builders	Capabilities where APIs exist but no MCP/A2A wrapper does — gap as signal, with a live "Recent verifications" feed	Free
/categories/{cluster}	Browser	Cluster view: known/listed, tested, runnable counts	Free
Homepage live-evidence strip	All visitors	4 tiles backed by Postgres: total benchmark runs, verifications (7d), scout runs (24h), demand events (24h), plus the routable-cells count	Free
Pro tier — billing skeleton live	Power users + teams	Clerk-backed sign-in, Stripe checkout + webhook, workspaces + roles, saved-recipes round-trip; freshness alerts on the roadmap	TBD after early users

Why the recipe model beats the executor model

Executing on the customer's behalf means storing keys, writing one connector per provider, and absorbing PCI / SOC2 scope per integration. That model loses at internet scale.

DIMENSION	EXECUTOR MODEL	OUR RECIPE + MARKETPLACE MODEL
Per-connector engineering	\$5-30K each, ongoing	Zero — adapters are protocol-only (MCP / A2A / OpenAPI)
Connector zoo at year 3	200+ hand-coded	Index size grows with the ecosystem
Compliance surface	PCI / SOC2 per provider	Minimal — we hold no customer keys
Moat	"Most adapters"	"Most trusted index + benchmarks + recipes"
Market timing	Crowded, late	Empty, early

Our place: *upstream of Zapier* (we recommend, they execute), *adjacent to G2* (vendor-neutral trust for a different supply), *layered above Smithery* (they catalog; we rank, recipe, and benchmark).

TAM — three stacking customer pools

We sit upstream of three buyers: the developer/operator wiring agents into their host, the agent vendor seeking neutral discovery + benchmarks, and the workflow or LLM host that wants a turnkey trust layer inside its UI. Each pool funds a different phase of the product.

CUSTOMER POOL	WHOSE BUDGET PAYS	TAM (~2026, US\$)	CAGR THROUGH 2030	BOTTOMS-UP GROUNDING
Developers + operators building agent workflows	Dev productivity + AI API spend	\$10–20B	25–35%	Cursor + Claude Desktop + Cline reach ~2M paying dev seats; n8n + Zapier + Make are ~\$1B+ combined ARR; the AI-using slice is the fastest-growing segment of both
Agent vendors (MCP / A2A / AI-native / SaaS-with-AI) seeking neutral discovery + benchmarks	Marketing / RevOps / API-growth budgets	\$1–3B	40–60%	~2,000 commercially viable agent vendors today; G2-comparable trust spend runs \$1–5k/mo per vendor; supply doubles as MCP/A2A supply doubles
Workflow + LLM hosts wanting turnkey discovery / recipe / benchmark inside their UI	OEM / partnership budgets	\$0.5–2B	20–30%	6–10 host platforms (n8n, Zapier, Make, Cursor, Cline, Claude Desktop, Gemini, Continue, ChatGPT) at \$0.5–5M/yr OEM-fit ceiling each

The trust moat + the honest threat

Five compounding assets:

1. **Discovery freshness** — 17 scout sources spanning four channel classes (canonical MCP registries, third-party aggregators, package registries, community-curated lists), recurring upkeep, scout fleet keeps growing.
2. **Verification evidence** — trust tier ladder, MCP tool probes, vendor cross-refs.
3. **Benchmark history** — per-capability runs over time; takes years to replicate trust.
4. **Refusal demand signal** — every unmet goal becomes a public demand event.
5. **Future marketplace network effects** — vendors register because users are here; users come because evidence is real.

Honest competitive threats:

- **Frontier LLM vendors** (Anthropic, OpenAI, Google) launch curated marketplaces inside their host. Our defence: vendor-neutral, cross-host, multi-protocol.
- **Smithery, MCP Marketplace, official MCP Registry** are catalogs. We use them as inputs and add ranking + recipe + benchmark on top.
- **Zapier / n8n / Clay** execute or build workflows. We do not compete on execution. n8n is a partnership target, not a competitor.
- **AgentBench / Steel.dev** are research benchmarks. We benchmark for commercial routing, not papers.

Beachhead + first 90 days

Beachhead: developer agent workflows. Two advantages stack. **(1) Supply-side moat:** we already index the densest MCP/A2A supply on the market via a 17-scout discovery fleet that refreshes continuously and spans four distinct channel classes (canonical registries, third-party aggregators, package registries like npm, and community-curated awesome-* lists) — a new entrant has to rebuild the scout fleet across all four channels from scratch, or wait for the supply to settle (MCP supply is doubling quarterly; it will not). **(2) Adoption mechanic:** a single JSON merge into Claude Desktop's `claude_desktop_config.json` or Cursor's `.cursor/mcp.json` — no host integration required; n8n recipes import as standard workflow JSON. Dev community channels (HN, Reddit r/LocalLLaMA, MCP Discord, Cursor forum) carry adoption without paid distribution.

What is verified end-to-end today (May 2026): All five recipe formats are round-trip-verified. MCP-shape (Claude Desktop / Cursor) — emitted `mcpServers` entries spawn real npm-published MCP servers and complete the JSON-RPC `initialize` handshake (the same handshake Claude Desktop runs at startup; see `docs/manual-test-log.md`). HTTP-shape (n8n / CLI) — emitted nodes use registry-authoritative endpoint, method, and auth-header wiring, with an `npm view` resolvability gate before any MCP install command is emitted. The four pitch-critical browser flows (homepage live-evidence strip, agent-detail benchmark + verification panels, recent-verifications panels on the gap surfaces, and the `/goal` honest-refusal path with a gap-only runbook) are codified as a Playwright spec at `apps/web/tests/e2e/pitch-flows.spec.ts` (run via `make web-e2e`).

Next 90 days, in order:

1. **Ship public v0** at `www.planmyagents.com` on AWS (EC2 + RDS + CloudFront).
2. **Grow benchmark coverage.** A "cell" is one capability (e.g., `payment_authorization`) scored end-to-end against real provider sandboxes for pass/fail, quality, latency, and cost. Three cells are routable today (Razorpay live, Resend + Firecrawl fixture-backed — both flip live the moment vendor keys reach the cron host). Widen into the commodity sub-tasks that recur across goals — email verification (Hunter is already `capability_verified`), web search, contact enrichment, transactional email, shipping — and push the strongest cells into the `publishable` credibility band so the homepage shows ≥ 6 routable cells with at least 4 live (not fixture).
3. **Operate the evidence loop in production.** The cron is shipped — `make benchmark-cron`, `make discovery-verify-cron`, `make evidence-cron`, the `evidence_health` Postgres view, and a GitHub Actions guard that fails `main` when the live counters drop to zero. The 90-day job is to land it on the production host, alert on its failures, and watch the routable-cell count climb past 3.
4. **Onboard design partners.** Humans, not agents: developers and operators who already use Claude Desktop, Cursor, or n8n with MCP, run our recipes against their own credentials each week, and trade a short written debrief for early access and influence on the roadmap.
5. **First Tier-1 BD outreach** — n8n, Cursor, Anthropic. Target: one signed collaboration, not three.

What we are — and what we are not

	YES TODAY	DESIGNED / PLANNED	NOT US
Index agent supply (17 scouts across 4 channel classes)	✓		
Decompose goals + match providers	✓		
Honest refusal with reasons	✓		
Recipe exports (5 formats)	✓ all five round-trip-verified — MCP-shape via JSON-RPC <code>initialize</code> handshake; HTTP-shape (n8n / CLI) via registry-authoritative endpoint + method + auth + <code>npm view</code> gate		
Cost-capped, fail-closed execution	✓ (current capability set)	Broader generic MCP / A2A / OpenAPI execution	
Trust + benchmark evidence model	✓ (3 routable cells — 1 live + 2 response-fixture pending keys; live evidence strip on homepage; 60 verification records; 5 evidence tables in Postgres queried live; CI guard fails <code>main</code> on cron regression)	Many more live cells; flip fixture cells to live	
Vendor marketplace (claim, benchmark, sponsor)		✓ Phase 2	
Partner OEM into hosts		✓ Phase 3	
Hold customer credentials			✗ BYO credentials only
Hand-write per-vendor connectors at scale			✗ Protocol-only adapters
Workflow builder UI			✗ Users run recipes in their host of choice
Pay-to-rank marketplace			✗ Visibility is paid; ranking position is not
Universal executor that runs every task on every agent			✗ Honest refusal beats false execution

Founder + ask

Founder: Deepraj Jha. 12 years backend and platform engineering across Adobe, Amazon Payments, and Salesforce. Building the prototype solo today; looking for a complementary co-founder on GTM, BD, and company-building.

What I am asking for right now:

- **Design partners** — developers or operators (humans, not agents) who already use Claude Desktop, Cursor, or n8n with MCP and want better discovery, recipes, and benchmarks.
- **Vendor introductions** — to product/RevOps at Apollo, Hunter, Perplexity, Linkup, Tavily, Exa, Apify, Firecrawl, Browserbase, Resend.
- **Partner introductions** — to BD at n8n, Cursor, Anthropic.
- **Honest feedback** on the trust model, the refusal-with-reasons UX, and the benchmark methodology.

Funding will follow real design-partner traction and a meaningful slate of `publishable`-band benchmark cells. The deck is intentionally not a seed ask yet.

THANK YOU

PlanMyAgents

The universal trust and marketplace layer for AI agents

The agent supply is exploding across MCP, A2A, and AI-native services. The trust, ranking, recipe, and benchmark layer for it does not yet exist. We are building it — vendor-neutral, cross-host, evidence-first — in three phases: index and recommend, monetize the vendor side, then OEM into workflow and LLM hosts.

Deepraj Jha · jha.deepraj@gmail.com · planmyagents.com